University of Exeter

ECM2433 The C Family

C++ classes

struct in C and C++

Struct is the closest thing to a class in C.

```
struct Student {
      char name[50];
      int age;
};
```

functions must be external and accept the struct

```
void printStudent(struct Student s) {
      printf("%s is %d\n", s.name, s.age);
}
```

must use struct keyword

```
int main() {
      struct Student s1 = {"Alice", 20};
      printStudent(s1);
}
```

Struct is the “full-featured class” in C++

```
struct Student {
      char name[50];
      int age;

      void print() {
          cout << name << " is " << age << endl;
      }
};
```

can have member functions

no struct keyword needed, struct name is a type

```
int main() {
      Student s1(“Alice”, 20);
      s1.print(); // Object-oriented call
}
```

The only difference between `struct` and `class` in C++:

- **`class`**: members are private by default.
- **`struct`**: members are public by default.
- *(Use `struct` for simple data holders, use `class` for complex objects).*

Reminder : class declaration

University of Exeter

Stack.h

```
#define SIZE 100


// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        void    push(int);
        int     pop(void);
};
```

private class variables

public member function prototypes

Class constructors

```
#include <Stack.hpp>
int main ()
{
    Stack    s1;
    int      value;

    s1.push(23);
    s1.push(1);
    value = s1.pop();

    return 0;
}
```

constructor function **implicitly** called

Stack.h

```
#define SIZE 100

// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        Stack();
        void    push(int);
        int     pop(void);
};
```

The constructor initializes the state of newly created objects

Class destructors

```
#include <Stack.hpp>
int main ()
{
    Stack    s1;
    int      value;

    s1.push(23);
    s1.push(1);
    value = s1.pop();

    return 0;
}
```

destructor function **implicitly** called

constructors and destructors have no return type

Stack.h

```
#define SIZE 100
// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        Stack();
        ~Stack();
        void    push(int);
        int     pop(void);
};
```

Parameterised constructors

University of Exeter

```
class NewClass
{
    int  a, b;
public:
    NewClass(int i, int j) {   // parameterised constructor
        a = i;
        b = j;
    }

    void showValues() {
        std::cout << "a = " << a;
        std::cout << ", b = " << b << std::endl;
    }
};
```

```
#include <iostream>

int main()
{
    NewClass nc(3,4);
    nc.showValues();
    return 0;
}
```

```
a=3, b=4
```

Scope resolution operator ::

Defining member functions outside declaration

```
class NewClass
{
      int  a,b;
   public:
      NewClass(int,int);
      void showValues();
}
NewClass::NewClass(int i, int j)
{
   a = i;
   b = j;
}
void NewClass::showValues()
{
   std::cout << "a = " << a;
   std::cout << ", b = " << b << std::endl;
}
```

Scope resolution operator :: specifies the scope of member functions, classes and namespaces.

Pointers: this

this is a **pointer** to the current object. Used to avoid ambiguities:

```
class NewClass
{
        int  a,b;
    public:
        NewClass(int,int);
        void showValues();
}

NewClass::NewClass(int a, int b)
{
    this->a = a;
    this->b = b;
}
```

```
a = a;
b = b;
```

Parameterised constructors: overloaded

University of Exeter

```
class NewClass
{
        int  a,b;
    public:
        NewClass(int, int);     // parameterised constructor
        NewClass(int);          // parameterised constructor
        void showValues();
};
NewClass::NewClass(int i, int j)
{
    a = i;
    b = j;
}
NewClass::NewClass(int i)
{
    a = i;
    b = 0;
}
```

valid overloading: differing number of arguments

The compiler knows which one we want to use based on the inputs!

```
#include <iostream>

int main ()
{
    NewClass nc(3,4);
    nc.showValues();
    NewClass md(2);
    md.showValues();
    return 0;
}
```

```
a=3, b=4
a=2, b=0
```

Parameterised constructors: default values

```
class NewClass
{
      int  a,b;
   public
      NewClass(int, int);    // parameterised constructor
      void showValues();
};


NewClass::NewClass(int i, int j=0)
{
   a = i;
   b = j;
}
```

```
#include <iostream>

int main ()
{
   NewClass nc(3,4);
   nc.show();
   NewClass md(2);
   md.show();
   return 0;
}
```

```
a=3, b=4
a=2, b=0
```

Function parameters: default values

All functions may have default parameters:

- All parameters following a parameter with a default must also have default values

```
void NewClass::someFunction(int i,   int j=0)    ✓
void NewClass::someFunction(int i=1, int j=0)    ✓
void NewClass::someFunction(int i=1, int j)      ✗
```

Does not have to be a class member function:

```
void someOtherFunction(int a, char b='X', float f=0.0,
                       std::string str="Hello!")
```

Default constructors

```
class NewClass
{
      int  a,b;
   public
      void showValues();
}
```

```
NewClass::NewClass()  // default constructor
{
    a = 1;
    b = 0;
}
```

If we call our constructor without parameters we can set default ones

```
#include <iostream>

int main ()
{
   NewClass n;
   n.showValues();
   return 0;
}
```

a=1, b=0

Copy constructor

A special constructor that creates a new object as a copy of an existing object.

Copy constructor declaration (function prototype):

```
MyClass (const MyClass &)
```

Automatically called when:

- Initialising an object

  ```
  MyClass n1 = n2;
  ```

- Pass by value

  ```
  void doSomething(Myclass n)
  ```

- Return by value

  `Myclass getClass(… return n;)` The returned object is a copy.

- If the copy constructor is private, that prevents copying of the objects of that class

Copy constructor

If no copy constructor is provided, C++ makes a default one that does a member-wise copy (unlike Java):

```
class MyClass
{
    int      x;
    char     y;
    char    *str;
};

MyClass a;
...
MyClass b = a;
```

Memberwise copy:

```
b.x   = a.x;      // ok
b.y   = a.y;      // ok
b.str = a.str;    // probably not ok
```

Copy constructor

If no copy constructor is provided, C++ makes a default one that does a member-wise copy (unlike Java):

```
class MyClass
{
    int      x;
    char     y;
    char    *str;
};

MyClass a;
...
MyClass b = a;
```

Better copy constructor:

```
MyClass::MyClass (const MyClass& other){
    this->x = other.x
    this->char = other.y
    this->str = (char *)malloc(strlen(other.str)+1);
    if (!this->str) { … }
    strcpy(this->str, other.str);
}
```

```
b.x = a.x;
b.y = a.y;
b.str = (char *)malloc(strlen(a.str)+1);
if (!b.str) { … }
strcpy(b.str,a.str);
```

Copy constructor

If no copy constructor is provided, C++ makes a default one that does a member-wise copy (unlike Java):

```
class MyClass
{
    int      x;
    char     y;
    char    *str;
};

MyClass a;
...
MyClass b = a;
```

Even better copy constructor:

```
MyClass::MyClass (const MyClass& other){
    this->x = other.x
    this->char = other.y
    this->str = new char[strlen(other.str)]+1];
    strcpy(this->str, other.str);
}
```

```
b.x = a.x;
b.y = a.y;
b.str = new char[strlen(a.str)+1];
strcpy(b.str,a.str);
```

Destructor

By default, a destructor releases only the resources that the object itself owns; anything acquired manually must be released manually.

```
NewClass::~NewClass()
{
    // free/deallocate any data here

}
```

In our NewClass class we are not dynamically allocating any data by using the malloc or new

```
MyClass::MyClass()
{
    x = 1;
    y = 2;
    str = (char*)malloc(100*sizeof(char));
};
```

```
MyClass::~MyClass()
{
    // free/deallocate any data here
    free(str);
}
```

Summary

- The only difference between struct and class in C++ is that members are public by default in struct, while private by default in class.
- **Constructors** define how objects are initialised and ensure objects start in a valid state.
- **Destructors** define how objects are cleaned up and are responsible for releasing manually acquired resources.
- **this** is a pointer to the current object, used to explicitly access its data members and resolve name ambiguities.